Biomedical Image Classification with Knowledge Distillation

A project guide — what was built, why each decision was made, and how to defend it.

Repository: github.com/joshinitin2016883-sketch/medmnist-kd-reproducible · Original notebook by Akash Samanta (MIT) · Harness, experiments and analysis by Nitin Joshi

> **How to read this.** Every section is structured as **WHAT** the thing is, **WHY** it matters for this specific problem, and **HOW** it actually showed up in the measured numbers. Every figure here comes from a real run recorded in `results/`. Nothing is estimated.

1. The problem

The task

Classify dermatoscopic images of skin lesions into 7 diagnostic categories. The data is **DermaMNIST**, part of the MedMNIST v2 benchmark, derived from the HAM10000 dataset — 10,015 images collected in Vienna, Austria and Queensland, Australia.

Split	Images	Purpose
train	7,007	fit model weights
val	1,003	choose when to stop; fit any decision rule
test	2,005	touched once, at the end

The defining feature: severe class imbalance

Class	Meaning	Train	Test
nv	melanocytic nevi (benign mole)	4,693	1,341
mel	**melanoma (malignant)**	779	223
bkl	benign keratosis-like lesions	769	220
bcc	basal cell carcinoma	359	103
akiec	actinic keratoses / intraepithelial carcinoma	228	66
vasc	vascular lesions	99	29
df	dermatofibroma	80	23

`nv` is **67%** of the test split. `df` is **1.1%**. That ratio, roughly 58:1, drives nearly every design decision in this project.

2. Concepts you must be able to explain

2.1 Transfer learning

WHAT Start from a ResNet-50 already trained on ImageNet (1.2M natural images, 1000 classes), replace the final classification layer with a fresh 7-output layer, and continue training on dermoscopy images.

WHY 7,007 training images is far too few to learn good visual features from scratch for a 25.6-million-parameter network. But the early layers of any image classifier learn generic things — edges, textures, colour gradients — that transfer across domains. You are reusing that and only learning the domain-specific part.

HOW `models.resnet50(weights=IMAGENET1K_V1)`, then `model.fc = nn.Linear(2048, 7)`. Note this project fine-tunes *all* layers from step one — there is no frozen backbone. That is a defensible choice but not the only one, and "progressive unfreezing" remains an untested item in the backlog.

2.2 Why accuracy lies — the single most important idea here

WHAT Accuracy is the fraction of predictions that are correct. On imbalanced data it is dominated by the majority class.

> **Work this out yourself — it is the question you will be asked.**
>
> Replace the entire 25-million-parameter model with three characters: `return nv`. Predict "benign mole" for every single image, always.
>
> ```
> Accuracy   = 1341 / 2005                    = 0.6688   (67%)
>
> For class nv:  precision = 1341/2005 = 0.6688
>                recall    = 1341/1341 = 1.0000
>                F1        = 2PR/(P+R) = 0.8016
> All other classes: F1 = 0
>
> Macro-F1   = (0.8016 + 0 + 0 + 0 + 0 + 0 + 0) / 7 = 0.1145
> ```
>
> **67% accuracy. 0.11 macro-F1.** A model that has learned nothing, that cannot detect a single melanoma, scores two-thirds accuracy. That gap is why accuracy is close to meaningless on this dataset.

2.3 Macro-F1 vs weighted-F1

WHAT Both average per-class F1. **Weighted-F1** weights each class by its support (how many test examples it has). **Macro-F1** weights every class equally.

WHY Weighted-F1 has the same blind spot as accuracy: with `nv` at 67% of the split, it is mostly a measurement of `nv`. Macro-F1 gives `df` — 23 images — the same say as `nv`'s 1,341. If you care whether the model detects rare malignancies, macro is the only one of the three that notices.

HOW Measured on the seeded baseline:

```
weighted-F1  0.7533        macro-F1  0.5587        gap = 0.195
```

The original project reported the weighted figure. That is not wrong, but it flatters the model by roughly 0.20 and conceals the melanoma result entirely.

2.4 Three imbalance corrections, and why bundling them is a problem

Class weighting

WHAT Multiply each sample's loss by a per-class weight, so rare classes contribute more gradient.

HOW The recipe computes `balanced` weights, then damps them three ways: `** 0.5`, divide by mean, clamp to [0.1, 2.0]. Actual result:

```
df 1.873   vasc 1.684   akiec 1.110   bcc 0.884   bkl 0.604   mel 0.600   nv 0.245
```

> **Worth knowing:** only the square root does anything. It takes the 58:1 spread to √58 ≈ 7.6:1. Dividing by the mean is a pure rescale. And *no class ever reaches the 2.0 clamp* — the clamp is inert, dead code that looks like it is protecting you.
>
> Also note `mel = 0.600`: melanoma, the clinically critical class, gets *below-average* weight, because at 779 training images it is not rare enough to trigger the correction.

Focal loss

WHAT $(1 - p_t)^\gamma \cdot CE$. Scales down the loss on examples the model already gets right, so training focuses on hard ones.

WHY With 4,693 easy `nv` images, standard cross-entropy spends most of its gradient confirming what the model already knows. γ=1.5 suppresses that.

Label smoothing

WHAT Instead of training toward "100% class 3", train toward "90% class 3, 10% spread across the rest". Discourages overconfidence.

> **The design flaw in the original:** all three are applied at once, with label smoothing hardcoded *inside* the focal loss class. Nobody can say which is helping, which is doing nothing, and which is actively hurting — they cannot be varied independently. The rewritten `train.py` exposes `--label-smoothing` as a parameter (default 0.1, unchanged) so the three can finally be separated. That experiment has not been run yet.

2.5 Knowledge distillation

WHAT Train a small "student" network (EfficientNet-B0, 5.3M params) to imitate a large "teacher" (ResNet-50 / DenseNet-121, 25M / 8M params). The loss combines ordinary cross-entropy against true labels with KL-divergence against the teacher's softened output distribution.

```
Loss = α · CE(student, true labels) + (1−α) · T² · KL(student_T ‖ teacher_T)
       α = 0.7,  temperature T = 3.0
```

WHY The teacher's full probability distribution carries more information than a one-hot label. If the teacher says "80% melanoma, 15% nevus, 5% keratosis", it is telling the student that melanoma and nevus look similar — a relationship the hard label "melanoma" destroys. These are called *dark knowledge*. Temperature T softens the distribution to make those small probabilities visible.

2.6 Grad-CAM

WHAT Backpropagate a class score to the last convolutional layer, average the gradients spatially to get per-channel importance weights, take the weighted sum of activation maps, ReLU it. Produces a heatmap of which

image regions drove the prediction.

WHY A number without a reason is not usable in a medical context. **But be careful what you claim:** Grad-CAM shows *where* the model looked, not *whether it was right to*. HAM10000 contains ruler marks, ink annotations and vignetting that correlate with class — a model keying on those would still produce a confident-looking heatmap.

3. Measurement discipline — the actual contribution

This section is what separates this repository from the original notebook, and it is the part most worth talking about in an interview.

3.1 Seeding and reproducibility

WHAT Fixing the random number generators so a run can be repeated exactly — weight initialisation, data shuffling, augmentation choices.

WHY The original notebook set no seed and never called `torch.save`. Consequence: **its published numbers cannot be regenerated by anyone, including its author.** Rerunning gives different weights, and the weights vanish when the kernel exits. Nothing can be built on top of a result like that.

HOW `train.py` seeds Python, NumPy and Torch, and writes a checkpoint carrying its own metadata — architecture, dataset, class count, seed, hyperparameters, epochs trained. `evaluate.py` reads that metadata rather than trusting flags, and hard-errors if the checkpoint's class count disagrees with the dataset.

3.2 The noise floor — experiment E0

WHAT Train the identical configuration three times, changing only the random seed, and measure how much the result moves.

WHY Without this you cannot tell an improvement from luck. If macro-F1 wanders by ±0.04 between identical runs, then a technique that "gains 0.02" has demonstrated nothing at all.

Seed	macro-F1	weighted-F1	Accuracy
42	0.5224	0.7450	0.7332
1337	0.5944	0.7588	0.7451
2024	0.5594	0.7561	0.7392
mean ± sd	**0.5587 ± 0.0360**	0.7533 ± 0.0073	0.7392 ± 0.0060

Macro-F1 is 5–6× noisier than the metrics the original reported. Three consequences:

- The original single-seed baseline (0.5224) happened to be the *worst* of the three, sitting 0.036 below the mean.
- An unpaired single-run comparison needs a difference above roughly **0.07 macro-F1** to mean anything. Most published techniques are worth 0.01–0.03 — individually undetectable this way.
- The original README claimed a distilled student beat its teacher by **0.65 accuracy points**. The seed-only range on accuracy is **1.20 points**. The claimed effect is about half the noise of its own measurement.

Why the noise is so large

Class	n	seed 42	seed 1337	seed 2024	range
vasc	29	0.4602	0.8070	0.6067	**0.347**
df	23	0.3684	0.5333	0.3614	**0.172**
nv	1341	0.8736	0.8830	0.8726	0.010

`vasc` swings by 0.35 on 29 test images — a handful changing outcome. Macro-F1 weights that class equally with `nv`, so it inherits the instability of the smallest classes. This is a property of the benchmark, not a bug to fix.

3.3 Paired vs unpaired comparisons — the technique that rescues this

WHAT A **paired** experiment changes only the inference or decision procedure and reuses the *same trained checkpoints*. An **unpaired** one retrains, so weights differ.

WHY In a paired comparison, seed variance affects both sides identically and cancels. Effects an order of magnitude smaller than the 0.036 unpaired noise floor become visible.

HOW TTA is paired — same weights, four flipped views averaged at inference. It shows a reliable +0.017 that would be invisible in an unpaired test. Resolution is necessarily unpaired, but its effect (+0.21) is so large the distinction does not matter.

4. The experiments

0.5587	0.7733	0.7906	+0.2319
28px baseline macro-F1	224px macro-F1	224px + TTA	total gain

EXP-4 — Native 224px source data ADOPTED

WHAT MedMNIST ships DermaMNIST at **28×28** by default (18.8 MB). MedMNIST+ also publishes it at native **224×224** (1,041 MB). The pipeline was upsampling 28×28 images 8× to feed a 224px network.

WHY Upsampling adds no information. Dermoscopic diagnosis depends on fine structure — pigment networks, dots and globules, streaks, blue-white veil. None of it survives at 28×28. This is a ceiling on every result in the project, and no loss function can lift it.

Metric	28px	224px	Δ
macro-F1	0.5587 ± 0.0360	**0.7733 ± 0.0248**	**+0.2145**
weighted-F1	0.7533	0.8622	+0.1089
accuracy	0.7392	0.8589	+0.1197
macro AUC	0.9334	0.9771	+0.0437

> **+0.2145 macro-F1 — about six times the noise floor.** Cost: +24% training time (15.7 → 19.5 min) and a 1 GB download. **VRAM is unchanged**, because the tensors reaching the network were already 224×224 either way. The GPU was doing identical work; the extra information was simply being discarded before it got there.

Per-class, the pattern confirms the mechanism:

Class	n	28px	224px	Δ
bkl	220	0.4812	0.7572	+0.2760
akiec	66	0.4338	0.7082	+0.2744
df	23	0.4211	0.6904	+0.2693
vasc	29	0.6246	0.8813	+0.2567
bcc	103	0.5550	0.7838	+0.2288
mel	223	0.5191	0.6632	+0.1441
nv	1341	0.8764	0.9286	+0.0523

`nv` — separable by gross colour and shape — gains least. Classes needing fine texture gain 4–5× as much. That is exactly what the resolution hypothesis predicts, which is why the result is believable rather than just large.

> **The caveat you must state before anyone asks.** Melanoma gains *least* of the lesion classes (+0.144), and its recall is the least stable: 0.62 / 0.82 / 0.72 across seeds at 224px, versus 0.66 / 0.61 / 0.65 at 28px. One seed got *worse*. Resolution does not solve melanoma-vs-nevus, which is the genuinely hard discrimination and remains the weakest link.

EXP-1 / EXP-5 — Test-time augmentation KEPT

WHAT At inference, run each image four ways — original, horizontal flip, vertical flip, both — and average the softmax outputs.

WHY Dermoscopy has **no canonical orientation**. A lesion rotated 180° is the same lesion, so flips are genuinely label-preserving. That is not true of every modality — flipping a chest X-ray horizontally invents dextrocardia, a real medical condition. Knowing when a transform is valid is domain knowledge, not a generic trick.

	28px	224px
Δ macro-F1	+0.0171	+0.0174
paired *p*	0.119	0.081
positive seeds	3 / 3	3 / 3

Nearly identical at both resolutions, so the gains stack. TTA reduces variance in the decision; resolution adds information. Different mechanisms, independent effects. Cost: 4× inference time (about 9s → 37s on the full test split), no retraining, no extra VRAM.

Honest caveat: with n=3 the paired t-test does not reach conventional significance (p=0.081) and the 95% CI includes zero. Direction is consistent and the mechanism is sound, but this is *promising*, not proven.

EXP-2 / EXP-6 — Logit adjustment REJECTED

WHAT Subtract `τ · log(train prior)` from each log-probability before argmax, correcting for the fact that the model has absorbed the 67% `nv` prior.

WHY IT WAS EXPECTED TO WORK Macro-AUC was 0.93 while macro-F1 was 0.52. The model *ranks* classes well; the decision rule converts that ranking badly. That gap looked like free headroom.

RESULT Validation selected `τ ≈ 0` on nearly every seed — meaning the *unadjusted* rule was already best. Mean test Δ: +0.0012 at 224px, −0.0028 at 28px. **No effect.**

The honest explanation: the class-weighted focal loss already applies a prior correction during training. Applying it again at decision time double-counts. A negative result, logged rather than buried.

EXP-3 — Per-class decision weights REJECTED

WHAT Seven multiplicative weights, one per class, fitted by coordinate ascent to maximise macro-F1 *on the validation split*, then applied unchanged to test.

Resolution	validation Δ	test Δ
28px	+0.0566	+0.0033
224px	+0.0451	**−0.0105**

This is the cleanest demonstration of overfitting in the project. The rule gains +0.05 macro-F1 on validation *every single seed, at both resolutions* — and delivers nothing on test at 28px, and actively *harms* test at 224px.

Seven free parameters fitted against 1,003 validation images, where four classes have fewer than 60 examples. That is more than enough capacity to memorise validation noise. Anyone reporting the validation figure would be claiming a +0.05 improvement that does not exist.

Compare with EXP-2's *single* parameter, which gained nothing anywhere. Together they make the point precisely: the 7-parameter rule did not fail because threshold tuning is wrong in principle — it failed because it fitted noise.

Where the gain actually came from

Configuration	macro-F1 (3 seeds)
28px, argmax — original recipe	0.5587 ± 0.0360
224px, argmax	0.7733 ± 0.0248
224px + TTA	**0.7906 ± 0.0314**

+0.2319 total. 92% is input resolution, 8% is test-time augmentation. Neither changed the model, the loss, the optimiser or the schedule. Both *rejected* experiments were decision-rule changes — which says the same thing from the other side: the decision rule was never the bottleneck, and neither was the loss function.

5. Engineering decisions

5.1 Why train and evaluate share code

Preprocessing is defined once, in `evaluate.py`, and imported by `train.py` and `app.py`. If the evaluation transform drifted from the training transform by even the normalisation constants, every reported metric would be quietly wrong with no error message. This is the classic train/serve skew bug and the only reliable defence is a single definition.

5.2 Why the checkpoint carries metadata

A bare `state_dict` does not say which architecture or dataset produced it. With a dozen checkpoints from Phase 2, mislabelling one would silently corrupt the experiment log. The checkpoint stores model name, dataset, class count, seed and hyperparameters; `evaluate.py` reads them and refuses to run if the class count disagrees with the dataset.

5.3 Why metrics are written before plots

An early version wrote `metrics.json` *after* rendering the confusion matrix. When Windows Application Control blocked matplotlib's DLL, the whole run died and a completed evaluation was thrown away. Rendering is now wrapped and non-fatal; failures are recorded as warnings. **General principle: never let a presentation step destroy a computation step.**

5.4 Why uncomputable metrics are omitted, not zeroed

scikit-learn returns `NaN` (with a warning, not an exception) for macro AUC when a class has no positive examples. Writing that to JSON produces a literal `NaN`, which is invalid JSON. Such metrics are omitted and noted in a `warnings` array, and `json.dumps(allow_nan=False)` makes any non-finite value a hard error. A zero or a NaN in a results table is an invented number.

5.5 Docker: why `COPY requirements.txt` before `COPY .`

> Docker caches each instruction as a layer, and invalidates every layer *below* a change.
>
> The dependency install is the expensive step — roughly 200 MB of PyTorch. If you wrote `COPY . .` first, every source file would sit in a layer above the cache and below the install. Editing one character in `app.py` would invalidate that layer and everything after it, forcing a full reinstall of PyTorch **on every single build**.
>
> Copying only the dependency manifest first means the install layer depends on nothing but that manifest. Source edits invalidate one cheap layer near the end.
>
> **General rule: order instructions from least- to most-frequently-changing.**

5.6 Other container decisions worth knowing

- **Multi-stage build** — dependencies install into a virtualenv in a builder stage; only that virtualenv is copied forward. pip's cache and build scratch never ship.
- **Base image pinned by digest, not tag** — the same tag resolves to different bytes over time as it is rebuilt for security patches, so a tag-pinned build is not reproducible. The trade-off: patches must now be adopted deliberately rather than silently.

- **CPU build of PyTorch** — ~200 MB against ~2.6 GB for CUDA. Serving one image per request gains nothing from a GPU.
- **Separate serving requirements** — the API needs no scikit-learn, pandas, matplotlib or seaborn. Dropping them removes roughly 400 MB.
- **Non-root user** — if the app is compromised, the attacker inherits an unprivileged account rather than root inside the container.
- **Model mounted, not baked in** — checkpoints are ~90 MB and change more often than code. Mounting keeps image version and model version independent, so retraining does not require a rebuild.
- **gunicorn with `--preload`** — the Flask development server is single-threaded with no request timeouts. `--preload` loads the model once before forking so workers share it copy-on-write instead of each holding its own ~100 MB copy.

5.7 Three real bugs found in the original Grad-CAM

1. `register_backward_hook` is deprecated and fires unreliably on modules whose forward does anything non-trivial. Replaced with `register_full_backward_hook`.
2. Hooks were registered on every call and never removed, so repeated use *stacked* hooks on the same model and leaked memory. Now scoped to a context manager. There is a regression test asserting the hook count is unchanged after repeated calls.
3. `cam / cam.max()` divides by zero when ReLU zeroes every activation. Now guarded.

6. Interview questions, with answers

Your model gets 73% accuracy. Is that good?

No — and accuracy is the wrong question on this dataset. Predicting the majority class for every image gets 67%. My model's 73% accuracy corresponds to a macro-F1 of 0.56, and the all-`nv` baseline would score 0.11 macro-F1. Accuracy compresses that entire difference into six points.

You report macro-F1 improved from 0.56 to 0.79. How do I know that is real?

Because I measured the noise floor first. Three runs differing only by random seed span 0.5224–0.5944, a standard deviation of 0.036. The resolution change is +0.2145 — about six times that. For the smaller TTA effect (+0.017) I used a paired design, reusing the same checkpoints so seed variance cancels; it was positive on all three seeds, though at n=3 the p-value is 0.081 and I would not claim significance.

Two of your experiments failed. Why are they still in the repository?

Because a negative result is a result. The per-class threshold tuning gained +0.05 macro-F1 on validation every single time and lost 0.01 on test — that is a measured demonstration of overfitting with seven parameters against 1,003 images. Deleting it would leave the next person to rediscover it. It also directly informs what I would try next.

Why did higher resolution help so much? Isn't that just "more pixels, better result"?

The mechanism is checkable, and I checked it. If resolution were adding generic capacity, all classes would improve similarly. Instead `nv` — separable by gross colour and shape — gained 0.05, while classes requiring fine dermoscopic texture gained 0.23–0.28. That differential is what the hypothesis predicts. And crucially, GPU compute and VRAM were identical in both conditions: the network always received 224×224 tensors. The 28px pipeline was upsampling 784 pixels into 50,176.

Your project uses focal loss, class weighting, and label smoothing. Which one helped?

I don't know, and that is a real gap. They are applied simultaneously in the original recipe, with label smoothing hardcoded inside the focal loss class, so they cannot be varied independently. I refactored it so they can be — `--label-smoothing` is now a parameter — but I have not yet run that ablation. What I can say is that the loss function was not the bottleneck: the two changes that worked were data resolution and inference-time averaging, neither of which touches the loss.

Would you deploy this?

Absolutely not, and the model card says so explicitly. Three reasons beyond the obvious lack of regulatory clearance. First, melanoma recall is roughly 0.72 at best — it misses about one in four melanomas. Second, HAM10000 was collected in Austria and Australia, both predominantly light-skinned populations, so the model should be assumed not to generalise across skin tones and nothing in the data lets me measure that. Third, softmax outputs are uncalibrated, so the confidence numbers are not probabilities.

> **Why is `weights=None` safe in your evaluation script?**
>
> Because `load_state_dict` immediately overwrites every parameter from the checkpoint. Downloading ImageNet weights first would just waste bandwidth. I use `strict=True` so that any mismatch between the checkpoint and the constructed architecture raises rather than silently leaving randomly-initialised layers in the model — which would produce plausible-looking but meaningless metrics.

> **Early stopping uses validation loss. Is that the right choice?**
>
> No, and I have evidence it is not. On every seed I examined, the epoch with the best validation macro-F1 was rejected in favour of the epoch with the best validation loss — giving up roughly 0.019 macro-F1 each time. Validation loss is dominated by `nv` at 67% of the split, so minimising it optimises something other than the target metric. I have implemented `--early-stop-metric val_macro_f1` but have not yet run it as a controlled experiment, so I am not claiming a gain.

7. What is still not done

Being able to state this precisely is worth more than pretending it is complete.

- **Six backlog experiments unrun** — cosine LR schedule, loss-component ablation, discriminative learning rates with progressive unfreezing, class-weight power sweep, checkpoint ensembling/SWA, mixup and cutmix. Each needs three seeds at ~20 minutes to clear the noise floor.
- **Early-stop-on-macro-F1** — implemented behind a flag, never run.
- **The Docker image has never been built.** The Dockerfile is written and reasoned through, but the daemon was unavailable, so no claim is made that it builds.
- **Possible lesion-level leakage is unverified.** HAM10000 holds ~10,015 images of ~7,500 distinct lesions. If MedMNIST split by image rather than by lesion, near-duplicate views of one lesion could straddle train and test and inflate every score here. This has not been checked and should be, before quoting any number as generalisation performance.
- **No calibration and no demographic evaluation** — bias is unmeasured, not absent.

> **The one-sentence version of this project:** the original work tuned loss functions inside an artificial 28×28 constraint that cost 0.21 macro-F1 — more than any of those techniques could plausibly recover — and the way that was established was by measuring the noise floor first, so that real effects could be told apart from luck.

All figures produced by `train.py` / `evaluate.py` / `tune_thresholds.py` and recorded in `results/`. Hardware: RTX 4050 Laptop, 6 GB. Harness, experiments and analysis by Nitin Joshi.